

PAU10

Implémentation d'un système de commande pour stabiliser la position en lacet pour un système aéronautique à deux rotors

Séances de travaux pratiques évaluées par
Ionela PRODAN

ionela.prodan@lcis.grenoble-inp.fr

2026

L'objectif de ce projet est de concevoir et de mettre en œuvre un système de commande pour stabiliser la position en lacet d'un système aéronautique à deux rotors. Cela implique de comprendre le fonctionnement des capteurs et des actionneurs, d'analyser leurs signaux de sortie et de mettre en place un filtrage pour améliorer la précision des mesures. Ensuite, un correcteur de type PID et ses variantes sera conçu pour stabiliser la position en lacet à un point de consigne. Le système de contrôle sera d'abord développé et testé en simulation à l'aide d'un jumeau numérique créé dans Matlab, permettant ainsi de valider le comportement du demi-quadcoptère dans un environnement virtuel. Ensuite, le contrôleur sera transféré et mis en œuvre sur le matériel réel via une interface entre Matlab et le système physique. Enfin, le contrôleur sera implémenté sur un microcontrôleur Arduino, permettant de contrôler directement la position en lacet et d'évaluer la mise en œuvre de l'algorithme de commande en dehors de Matlab.

Ce projet utilise les outils et concepts issus des cours : i) AU330 (Traitement du signal); ii) AU360 (Systèmes linéaires asservis); IN330 (Algorithmique et programmation).



Contents

1. Actionneurs, capteurs et mesures	4
1.1. Présentation des actionneurs et des capteurs	4
1.2. Logiciel QUARC	4
1.3. Mesures de la position de lacet et du taux de lacet	5
1.3.1. Configuration d'un modèle Simulink pour le Quanser Aero	6
1.3.2. Pilotage du Quanser Aero	7
1.3.3. Mesure de l'angle de lacet à partir de l'encodeur	8
1.4. Filtrage et estimation de la vitesse angulaire de lacet	8
1.4.1. Rappel filtrage passe-bas et passe-haut	9
1.4.2. Estimating the yaw rate from the encoder via high-pass filtering	9
1.4.3. Mesure du taux de lacet à partir du tachymètre pour une entrée constante	12
2. Commande pour stabiliser la position en lacet pour un système à deux rotors	14
2.1. Identification d'un modèle à fonction de transfert à partir d'une expérience de réponse indicielle	15
2.1.1. Modèle de la fonction de transfert	15
2.1.2. Modélisation par réponse du premier ordre	15
2.1.3. Déterminer la fonction de transfert du Quanser Aero	17
2.2. Analyse de la stabilité du modèle	19
2.3. Conception et implementation d'un correcteur de type PID	20
2.3.1. Rappel	20
2.3.2. Implémenter un correcteur de type PID pour le système Aero	21
2.3.3. Test de robustesse du correcteur PID de l'Aero	22
2.4. Conception d'un correcteur par placement des pôles de la boucle fermée, structure RST	23
3. Implémentation de la commande sur Arduino	24
3.1. Préparation	24
3.2. Discrétisation du correcteur	24
3.3. Programmation du correcteur	25
3.3.1. Explication du code	25
3.3.2. Implémentation du contrôleur PID sur Arduino	26
A. Annexes	28
A.1. Arduino code	28



Figure 1: Quanser Aero

Le Quanser Aero, illustré à la Fig. 1, est un système aéronautique compact à deux degrés de liberté doté de deux rotors, qui peut être utilisé pour réaliser une variété d'expériences basées sur des actionneurs et le contrôle de vol. Le Quanser Aero peut être configuré avec les modules d'interface :

- **QFLEX 2 USB** qui permet la commande par ordinateur via une connexion USB;
- **QFLEX 2 Embedded** qui permet la commande par un microcontrôleur tel qu'un Arduino via une interface SPI à 4 fils.

Pour toutes les versions, le système est entraîné par deux moteurs brushless à courant continu (CC) de 18V à entraînement direct. Les moteurs sont alimentés par un amplificateur Pulse Width Modulation (PWM) intégré avec détection de courant intégrée. Des encodeurs rotatifs à sortie simple sont utilisés pour mesurer la position angulaire des moteurs à courant continu, et la vitesse des moteurs peut être mesurée à l'aide d'un tachymètre logiciel.

Le manuel d'utilisation et la documentation de démarrage rapide sont disponibles en téléchargement [ici](#).

1. Actionneurs, capteurs et mesures

1.1. Présentation des actionneurs et des capteurs

Actionneurs : Le Quanser Aero est équipé de deux moteur brushless à courant continu, connectés à un amplificateur PWM. Pour plus de détails, veuillez consulter le manuel d'utilisation du Quanser Aero.

Capteurs :

- Les **encodeurs** utilisés pour mesurer : i) **le tangage du corps de l'Aero et la position angulaire des moteurs à courant continu** sur le Quanser Aero sont des encodeurs à arbre optique à sortie simple. Ils délivrent 2048 impulsions par révolution en mode quadrature (512 lignes par révolution); ii) **le lacet du support de fourche** sur le Quanser Aero sont des encodeurs optiques. Ils délivrent 4096 impulsions par révolution en mode quadrature (1024 lignes par révolution). Pour un rappel utile sur les encodeurs en général, vous pouvez regarder cette vidéo : youtu.be/SCR5YJZVyKg?si=42JR-znCino9vCTs.
- Les **tachymètres** (compteurs de révolutions, tachys, compte-tours, jauges de RPM) sont des instruments qui mesurent la vitesse de rotation d'un arbre ou d'un disque, comme dans un moteur ou une autre machine. Ces appareils affichent généralement le nombre de révolutions par minute (RPM) sur un cadran analogique calibré, mais les affichages numériques sont de plus en plus courants.

L'interaction entre les différents composants matériels du Quanser Aero est illustrée à la Fig. 2. À partir de cette figure, on peut observer que les signaux des capteurs et des actionneurs interagissent avec l'ordinateur via la carte d'acquisition de données, communément appelée carte DAQ, où les signaux sont convertis du domaine analogique au domaine numérique.

1.2. Logiciel QUARC

Le logiciel QUARC de Quanser sera utilisé pour interagir avec le matériel du système Aero ainsi qu'avec le système dans l'environnement virtuel Quanser Interactive Labs. QUARC permettra de commander le système half-quad et de lire la position en lacet ainsi que la vitesse angulaire en lacet.

Les étapes de base pour créer un modèle Simulink avec QUARC afin d'interagir avec le matériel du Aero via la carte d'acquisition de données (DAQ) sont les suivantes:

- Créer un modèle Simulink qui interagit avec la DAQ intégrée au Aero en utilisant des blocs de la bibliothèque QUARC Targets.
- Générer le code temps réel.
- Exécuter le code.

Si vous souhaitez en savoir plus sur QUARC :

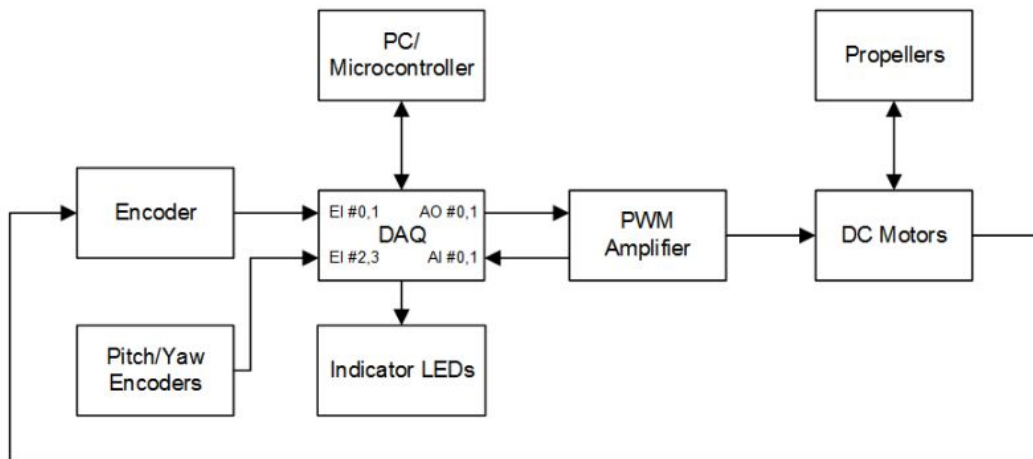


Figure 2: Interaction entre les différents composants matériels du Quanser Aero.

- Suivez le lien ci-dessous pour regarder la vidéo intitulée Getting Started with QUARC : www.quanser.com/tutorial/quarc-essentials-hardware-interfacing/
- Lorsque nécessaire, vous pouvez également taper doc quarc dans Matlab pour accéder à la documentation et aux démonstrations de QUARC.

1.3. Mesures de la position de lacet et du taux de lacet

L'objectif ici est de construire un modèle Simulink en utilisant des blocs QUARC pour piloter l'Aero, puis mesurer l'angle de lacet correspondant, comme illustré dans la Figure 3. Nous commencerons avec le Quanser Aero dans l'environnement virtuel des laboratoires interactifs de Quanser, avant d'utiliser le système réel.

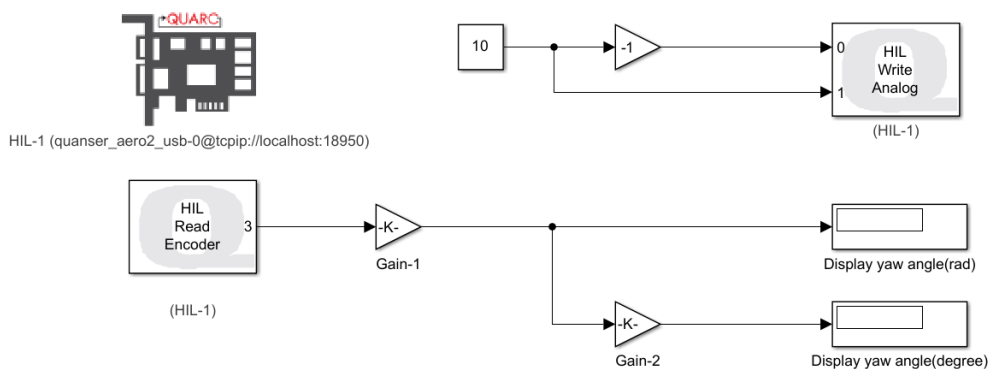


Figure 3: Modèle Simulink utilisé avec QUARC pour piloter le moteur à courant continu et lire l'angle sur le Quanser Aero.

1.3.1. Configuration d'un modèle Simulink pour le Quanser Aero

Suivez les étapes ci-dessous pour construire un modèle Simulink qui interagira avec le jumeau numérique de l'Aero en utilisant QUARC :

1. Ouvrez **Quanser Interactive Labs**, connectez-vous avec votre compte personnel. Choisissez le modèle approprié ; dans cette section, sélectionnez le modèle **Aero**
2. Vous pouvez voir que la LED du système est rouge. Dans **Options**, activez uniquement **Lock pitch**.
2. Ouvrez **Matlab**, créez un nouveau fichier **Blank Simulink**.
3. Ouvrez le **Library Browser**, développez l'élément **QUARC Targets** et accédez au dossier **Data Acquisition** → **Generic** → **Configuration**, comme illustré dans la Figure 4.

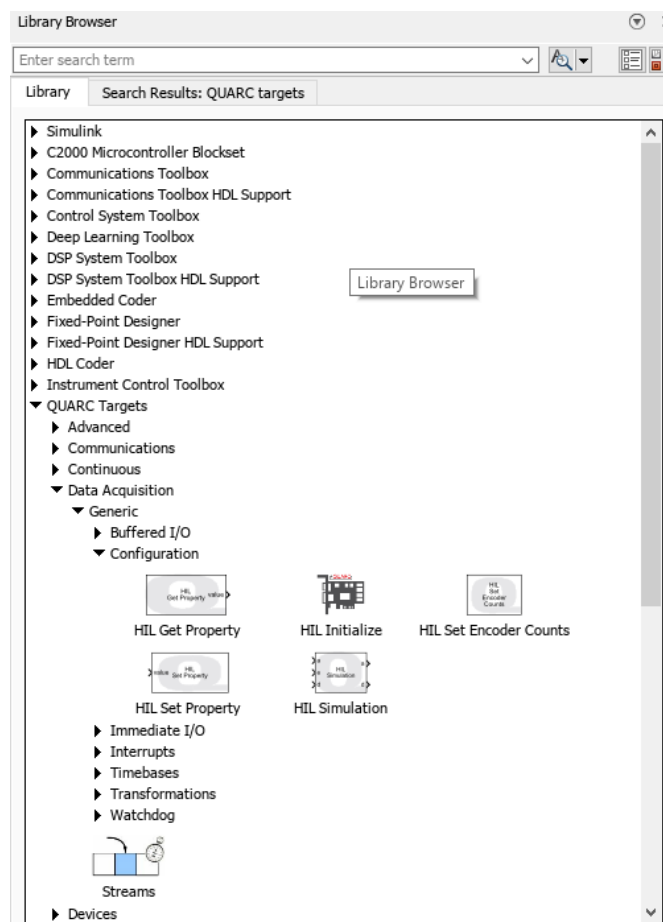


Figure 4: Library Browser

4. Cliquez et faites glisser le bloc **HIL Initialize** depuis la fenêtre de la bibliothèque

vers le modèle Simulink vide. Ce bloc est utilisé pour configurer votre dispositif d'acquisition de données.

- Double-cliquez sur le bloc **HIL Initialize**, dans la zone **Board type**, choisissez **quanser_aero2_usb**. Dans la zone **Board identifier**, tapez **0@tcpip://localhost:18950** pour vous connecter au servo virtuel.
- Dans la fonction **bar**, choisissez **QUARC** → **QUARC targets** → **quarc_win64.tlc** comme dans la Figure 5.

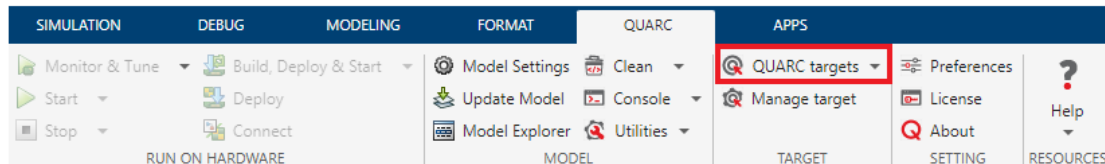


Figure 5: Quarc targets

- Dans la fonction **bar**, choisissez **HARDWARE** → **Monitor & Tune** comme dans la Figure 6. Si la LED du servo devient verte, votre connexion est réussie.

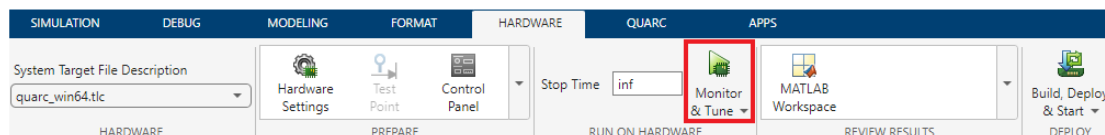


Figure 6: Monitor

1.3.2. Pilotage du Quanser Aero

Suivez les étapes ci-dessous pour piloter le jumeau numérique Quanser Aero :

- En utilisant le modèle Simulink que vous avez configuré dans la section précédente, ajoutez le bloc **HIL Write Analog**. Ce bloc est utilisé pour envoyer un signal depuis les canaux de sortie analogique #0 et #1 du dispositif d'acquisition de données. Ceux-ci sont connectés à l'amplificateur PWM embarqué qui pilote le moteur DC. Ce bloc se trouve également dans le **Library Browser** sous **QUARC Targets** → **Data Acquisition** → **Generic** → **Immediate I/O category**.
- Double-cliquez sur le bloc **HIL Write Analog**, tapez **[0 1]** dans la zone **Channels**.
- Nous allons alimenter l'Aero avec une tension de -10 V pour le rotor 0 et une tension de 10 V pour le rotor 1 afin que l'Aero puisse fonctionner dans l'environnement virtuel. Connectez le bloc **HIL Write Analog** à un bloc **constant** et à un bloc **gain** comme montré dans la Figure 3 (sans le bloc **HIL Read Encoder**, qui sera ajouté plus tard). Changez la valeur du bloc **constant** à 10.

4. Cliquez sur **Monitor & Tune** pour compiler et exécuter le fichier Simulink.
5. Assurez-vous que l'Aero tourne dans le sens antihoraire lorsque la tension appliquée est positive. Changez la valeur du bloc constant à -10 pour vérifier le changement de sens de rotation de l'Aero.
6. Arrêtez le fichier Simulink.

1.3.3. Mesure de l'angle de lacet à partir de l'encodeur

Suivez les étapes ci-dessous pour mesurer l'angle de lacet à partir de l'encodeur :

1. En utilisant le modèle Simulink que vous avez configuré dans la section précédente, ajoutez le bloc **HIL Read Encoder**. Ce bloc se trouve également dans le **Library Browser** sous **QUARC Targets** → **Data Acquisition** → **Generic** → **Immediate I/O category**.
2. Double-cliquez sur le bloc **HIL Read Encoder**, tapez **3** dans la zone **Channels**.
3. Connectez le bloc **HIL Read Encoder** à un bloc **Gain** et à un bloc **Display** comme dans la Figure 3. Dans le **Library Browser**, vous pouvez trouver le bloc **Display** sous **Simulink** → **Sinks** et le bloc **Gain** sous **Simulink** → **Math Operations**.
4. Dans le bloc **Gain 1**, ajoutez l'équation pour convertir les impulsions de l'encodeur en radians :

$$angle_rad = encoder_count \left(\frac{2\pi}{1024 \times 4} \right) \quad (1)$$

5. Dans le bloc **Gain 2**, ajoutez l'équation pour convertir les radians en degrés :

$$angle_deg = angle_rad \left(\frac{360}{2\pi} \right) \quad (2)$$

6. Cliquez sur **Monitor & Tune** pour compiler et exécuter le fichier Simulink. Lisez la mesure depuis les blocs **display**.
7. Arrêtez le fichier Simulink.

1.4. Filtrage et estimation de la vitesse angulaire de lacet

Cette section couvre les notions fondamentales des filtres passe-bas et passe-haut, l'estimation du taux de lacet à l'aide d'un encodeur, ainsi que la mesure directe de la vitesse angulaire via un tachymètre.

1.4.1. Rappel filtrage passe-bas et passe-haut

Un **filtre passe-bas** peut être utilisé pour éliminer les composantes haute fréquence d'un signal. La fonction de transfert de Laplace d'un filtre analogique passe-bas du premier ordre a la forme suivante :

$$H_{LP}(p) = \frac{\omega_f}{p + \omega_f} \quad (3)$$

où p est la variable de la transformée de Laplace et ω_f est la fréquence de coupure du filtre en rad/s. Toutes les composantes du signal ayant une fréquence supérieure seront atténuées d'au moins -3 dB (soit environ 70 % en amplitude).

Un **filtre passe-haut** peut être utilisé pour approximer ou estimer la dérivée temporelle d'un signal. La fonction de transfert de Laplace d'un filtre analogique passe-haut du premier ordre a la forme suivante :

$$H_{HP}(p) = \frac{\hat{\Omega}_f(p)}{\Theta(s)} = \frac{\omega_f p}{s + \omega_f} \quad (4)$$

Notez que le filtre passe-haut peut être vu comme la cascade d'une dérivée pure (qui prend la forme de s dans le domaine de Laplace) et d'un filtre passe-bas, comme montré dans la Figure 7.

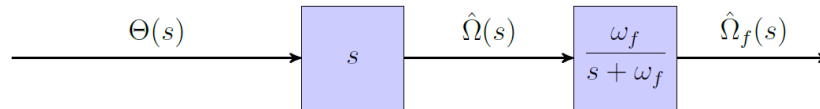


Figure 7: High-pass filter seen as a cascaded pure derivative and low-pass blocks

Ce filtre passe-haut peut être très utile pour fournir une estimation filtrée en passe-bas de la vitesse angulaire de lacet $\hat{\theta}_f(t) = \hat{\omega}_f(t)$ à partir d'une position de lacet mesurée $\theta(t)$.

1.4.2. Estimating the yaw rate from the encoder via high-pass filtering

À partir du modèle Simulink développé dans la section précédente, l'objectif est maintenant de construire un modèle Simulink qui estime le taux de lacet (ou la vitesse) de l'Aero en utilisant la mesure de la position de lacet fournie par l'encodeur pour une entrée constante (échelon), comme illustré dans la Figure 8.

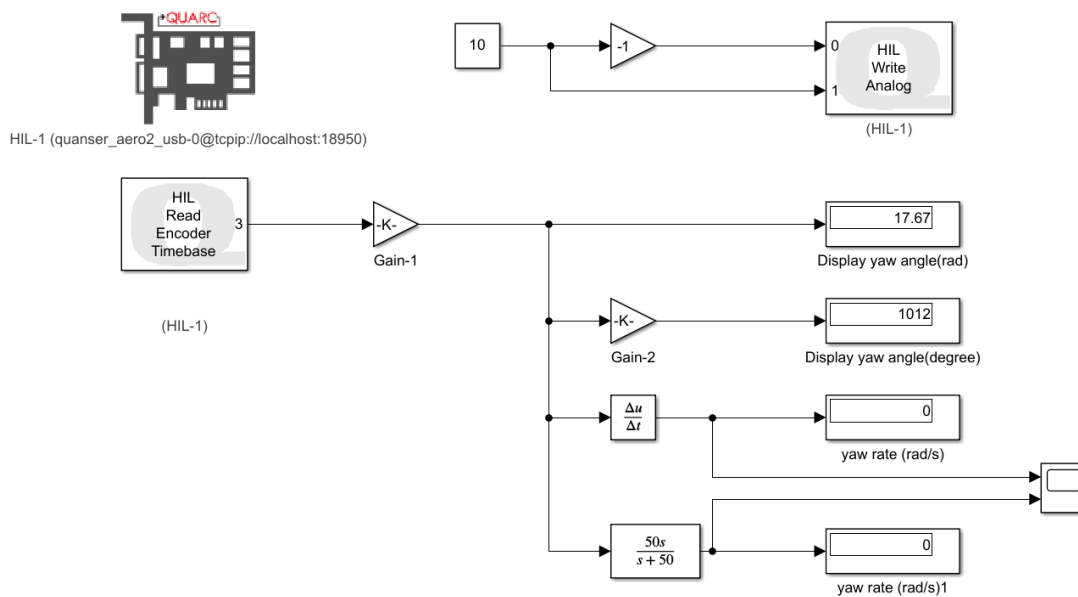


Figure 8: Estimation du taux de lacet à partir de la mesure de la position de lacet fournie par l'encodeur lorsque l'entrée est une valeur constante

1. Ouvrez le fichier Simulink que vous avez développé dans la section précédente. Nous allons calculer le taux de lacet en rad/s. Remplacez le bloc **HIL Read Encoder** par un bloc **HIL Read Encoder Timebase**. Changez le canal pour le canal 3. Vous pouvez trouver ce bloc dans la catégorie **QUARC Targets** → **Data Acquisition** → **Generic** → **Timebases**.
2. Utilisez la formule de la dérivée pour calculer le taux de lacet à partir de la position de l'angle de lacet. Choisissez le bloc **Derivative** comme dans la Figure 8 et connectez-le à la sortie du gain d'étalonnage de l'encodeur pour estimer le taux de lacet à l'aide de l'encodeur (en rad/s). Connectez la sortie du bloc **Derivative** à un bloc **Display** et un **Scope**.
3. Double-cliquez sur le bloc **Scope** pour modifier l'apparence et le comportement du bloc scope. Sélectionnez l'option **View** → **Configuration Properties** → **Time** pour configurer le bloc scope comme montré dans la Figure 9. Pour plus d'informations sur la configuration du bloc scope, tapez dans la fenêtre de commande Matlab : `doc TimeScopeConfiguration`.

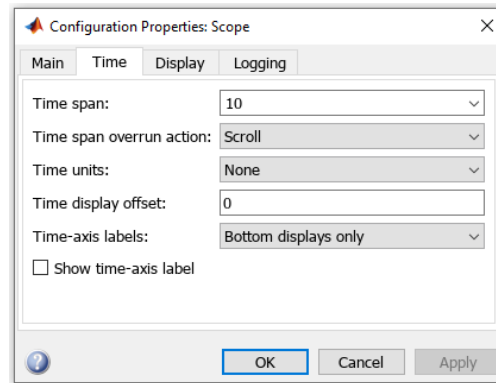


Figure 9: Fenêtre des paramètres de configuration pour configurer l'apparence du bloc **Scope**

4. Pour l'instant, n'incluez pas le bloc filtre. Réglez la valeur du bloc **constant** à 10.
5. Sélectionnez l'option **HARDWARE** → **Monitor & Tune** pour compiler et exécuter le fichier Simulink. Analysez les réponses de l'angle de lacet et du taux de lacet estimé.
6. Examinez la mesure de l'angle de lacet de l'encodeur à l'aide d'un nouveau bloc **Scope**. Effectuez un zoom sur la réponse de la position angulaire. Vous devriez observer que la mesure de la position angulaire n'est pas lisse ni continue. Elle est en escalier, c'est-à-dire segmentée en petits pas. Rappelez-vous que ce signal en escalier est la valeur d'entrée de la dérivée. La différentiation de ces petits pas entraîne de grandes valeurs dans la réponse, visibles comme des composantes haute fréquence.
7. Une façon d'atténuer les composantes haute fréquence dans un signal est d'appliquer un filtrage passe-bas. Pour obtenir une meilleure estimation de la vitesse angulaire à partir de la position angulaire, une approche consiste à placer un filtre passe-bas de la forme $\frac{50}{p+50}$ en cascade avec le composant dérivé $\frac{du}{dt}$ (qui correspond à une fonction de transfert p dans le domaine de Laplace). Notez qu'il est préférable, dans Simulink, d'implémenter les deux blocs en cascade dans un seul bloc de fonction de transfert $\frac{50p}{p+50}$, ce qui correspond à un filtre dit passe-haut.
Ajoutez un bloc **Transfer Fcn** et connectez-le au bloc **Scope** et au bloc **Display**. Réglez le bloc **Transfer Fcn** à $\frac{50p}{p+50}$, comme illustré dans la Figure 8.
8. Compilez et exécutez le fichier Simulink. Observez la réponse de la vitesse angulaire filtrée en passe-haut et basée sur l'encodeur. A-t-elle été améliorée ?
9. Quelle est la fréquence de coupure du filtre passe-bas $\frac{50p}{p+50}$? Donnez votre réponse en rad/s et en Hz.

10. Faites varier la fréquence de coupure, ω_f , entre 10 et 200 rad/s (soit entre 1,6 et 32 Hz). Quel effet cela a-t-il sur la réponse filtrée ? Considérez les avantages et les compromis liés à la diminution ou à l'augmentation de la fréquence de coupure du filtre.
11. Arrêtez le fichier Simulink.

1.4.3. Mesure du taux de lacet à partir du tachymètre pour une entrée constante

À partir du modèle Simulink développé dans la section précédente, l'objectif est maintenant de modifier ce modèle pour ajouter la mesure du taux de lacet de l'Aero obtenue par le capteur tachymètre et la comparer au taux de lacet estimé calculé à partir de l'encodeur. L'entrée envoyée à l'Aero reste une entrée constante. Construisez le modèle Simulink comme illustré dans la Figure 10.

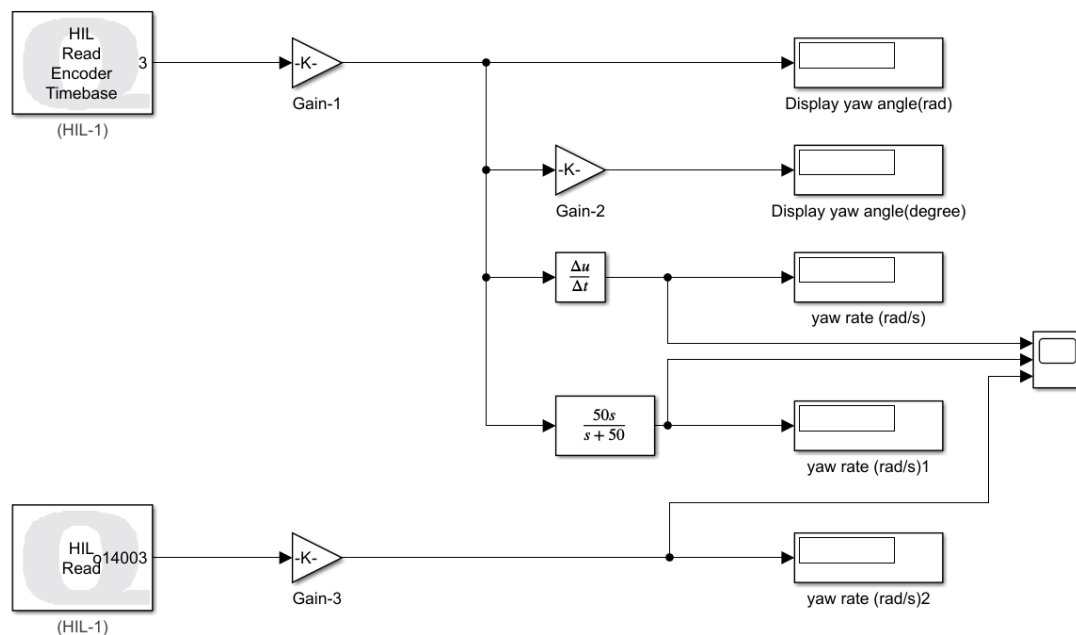


Figure 10: Modèle Simulink pour ajouter la mesure du taux de lacet de l'Aero

1. Ouvrez le fichier Simulink que vous avez développé dans la section précédente.
2. Ajoutez un bloc **HIL read**. Pour lire la mesure du tachymètre, nous lisons le canal 14003. Double-cliquez dessus et configurez-le pour qu'il affiche la mesure du tachymètre, comme montré dans la Figure 11.

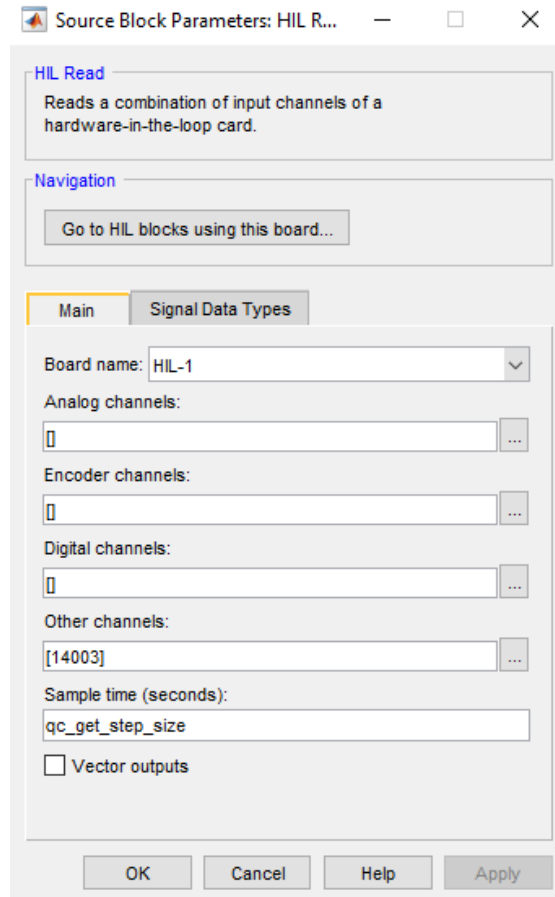


Figure 11: Configuration du bloc **HIL read** pour obtenir la vitesse angulaire de lacet à partir du capteur tachymètre

3. Ajoutez un bloc de calibration **Gain** avec la même valeur numérique que pour l'encodeur. Connectez la sortie du gain au **Scope** et au **Display**.
4. Sélectionnez l'option **Monitor & Tune** pour compiler et exécuter le code. Examinez les réponses de la vitesse basées sur l'encodeur filtré en passe-haut et sur le tachymètre. Semblent-elles similaires ?
5. Arrêtez le fichier Simulink.

2. Commande pour stabiliser la position en lacet pour un système à deux rotors

Le diagramme des forces appliquées (free-body diagram) du Quanser Aero en configuration demi-quadrotor est présenté dans la Figure 12.

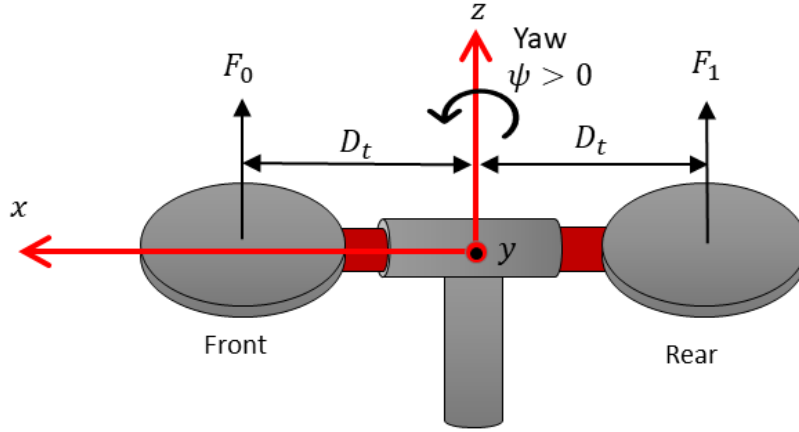


Figure 12: Aero 2 Half-Quadrotor model

Le couple produit par les rotors provoque la rotation du système autour de l'axe de lacet. L'équation linéaire du mouvement suivante décrit le mouvement de lacet, ψ , en fonction du couple appliqué :

$$J_y \ddot{\psi} + D_y \dot{\psi} = \tau_y \quad (5)$$

où J_y est le moment d'inertie autour de l'axe de lacet, D_y est le coefficient d'amortissement visqueux autour de l'axe de lacet, et τ_y est le couple agissant autour de l'axe de lacet. Comme l'axe de tangage est verrouillé et que seuls les mouvements de lacet sont pris en compte, la même tension est appliquée aux deux moteurs. L'équation peut être redéfinie en fonction de la tension appliquée aux deux rotors comme suit :

$$J_y \ddot{\psi} + D_y \dot{\psi} = D_t K_f u \quad (6)$$

où K_f est le gain de la force de poussée, D_t est la distance entre le pivot et le centre de l'hélice, et u est la tension moteur appliquée au moteur 0 et au moteur 1 des rotors. L'application d'une tension positive $u > 0$ entraîne une réponse positive en lacet, $\dot{\psi} > 0$.

Notez que, pour réaliser cela sur le matériel réel, l'entrée de contrôle est définie comme $V_0 = -u$ et $V_1 = u$, où V_0 est la tension appliquée au moteur 0 (le moteur avant), et V_1 est la tension appliquée au moteur 1 (le moteur arrière).

2.1. Identification d'un modèle à fonction de transfert à partir d'une expérience de réponse indicielle

2.1.1. Modèle de la fonction de transfert

À noter que dans ce projet, nous allons fixer le tangage et ne contrôler que la position en lacet du système.

À partir de l'équation du mouvement, le modèle de fonction de transfert du système aéronautique à deux rotors est:

$$J_y \left(\Psi(p)p^2 - \Psi(0)p - \dot{\Psi}(0) \right) + D_y(\Psi(p)p - \Psi(0)) = D_t K_f U(p) \quad (7)$$

Étant donné que le système part du repos, les conditions initiales sont nulles, c'est-à-dire $\Psi(0) = 0$ et $\dot{\Psi}(0) = 0$ on obtient la fonction de transfert suivante du système:

$$\frac{\Psi(p)}{U(p)} = \frac{D_t K_f}{p(J_y p + D_y)}. \quad (8)$$

La fonction de transfert pour la vitesse en lacet peut être exprimée par:

$$\frac{\beta(p)}{U(p)} = \frac{D_t K_f}{J_y p + D_y}. \quad (9)$$

avec $\beta(t) = \dot{\Psi}(t)$.

2.1.2. Modélisation par réponse du premier ordre

La réponse mesurée de la vitesse angulaire de lacet à une entrée en échelon de tension peut être modélisée à l'aide d'une fonction de transfert du premier ordre. La réponse réelle est d'ordre supérieur, mais un modèle du premier ordre peut nous fournir une approximation permettant d'obtenir des paramètres assez précis.

Nous pouvons déterminer le **coefficient d'amortissement visqueux** D_y et le **gain de la force de poussée** K_f du modèle de fonction de transfert du taux de lacet du demi-quadrotor en mesurant le gain DC et la constante de temps de la réponse.

La réponse d'un système du premier ordre peut être décrite par la fonction de transfert suivante :

$$\frac{Y(p)}{U(p)} = \frac{K}{1 + Tp} \quad (10)$$

où K est appelé gain en régime permanent, ou gain DC, et T est la constante de temps. La réponse à un échelon montrée dans la Figure 13 est générée en utilisant cette fonction de transfert avec $K = 5$ rad/V.s et $T = 0.05$.

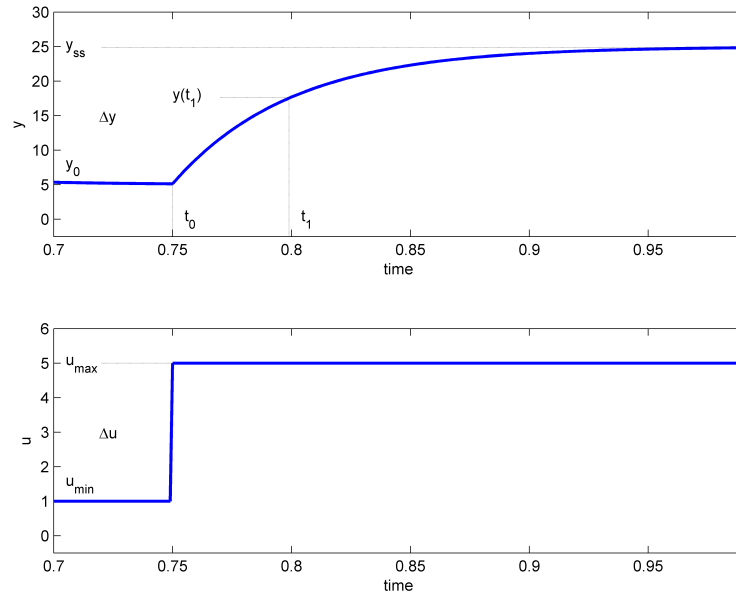


Figure 13: Signal d'entrée et de sortie utilisé dans la méthode de test par réponse à un échelon

L'entrée en échelon commence au temps t_0 . Le signal d'entrée a une valeur minimale u_{min} et une valeur maximale u_{max} . Le signal de sortie résultant est initialement à y_0 . Une fois l'échelon appliqué, la sortie tente de le suivre et se stabilise finalement à sa valeur en régime permanent y_{ss} . À partir des signaux d'entrée et de sortie, le gain en régime permanent est donné par :

$$K = \frac{y_{ss} - y_0}{u_{max} - u_{min}} = \frac{\Delta y}{\Delta u}. \quad (11)$$

La constante de temps d'un système T est définie comme le temps que met le système, après l'application d'une entrée en échelon, pour atteindre 63,2, % de sa valeur en régime permanent, c'est-à-dire, pour la Figure 13 :

$$t_1 = t_0 + T$$

avec

$$y(t_1) = 0.632\Delta y + y_0 \quad (12)$$

Ensuite, on peut lire le temps t_1 correspondant à $y(t_1)$ à partir des données de la réponse dans la Figure 13. À partir de cela, la constante de temps du modèle peut être déterminée comme suit :

$$T = t_1 - t_0. \quad (13)$$

À partir du modèle Aero dans l'environnement virtuel, déterminer les paramètres de la fonction de transfert.

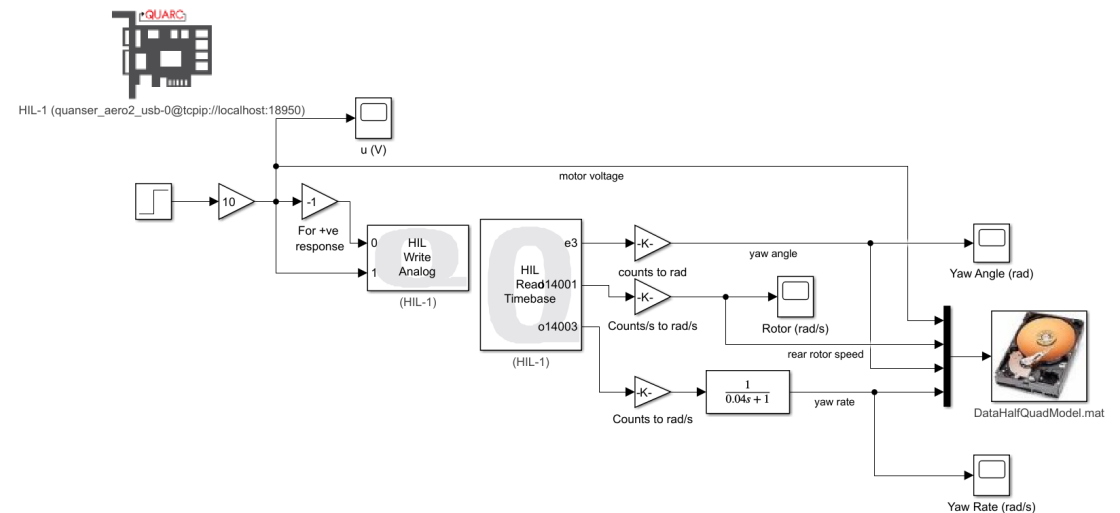


Figure 14: Modèle Simulink de la réponse en boucle ouverte.

1. Charger les paramètres du Quanser Aero, y compris, D_t, J_y . Ouvrir et exécuter le fichier Matlab :
`/Software/Quanser_Aero/Parameter_Estimation/aero2_parameters.m`
2. Ouvrir le fichier Simulink :
`/Software/Quanser_Aero/Parameter_Estimation/q_aero2_half_quad_model.slx`
3. Sélectionner l'élément **Monitor & Tune** pour compiler et exécuter le code. La réponse observée sur le **Scope** devrait être similaire à la Figure 15.

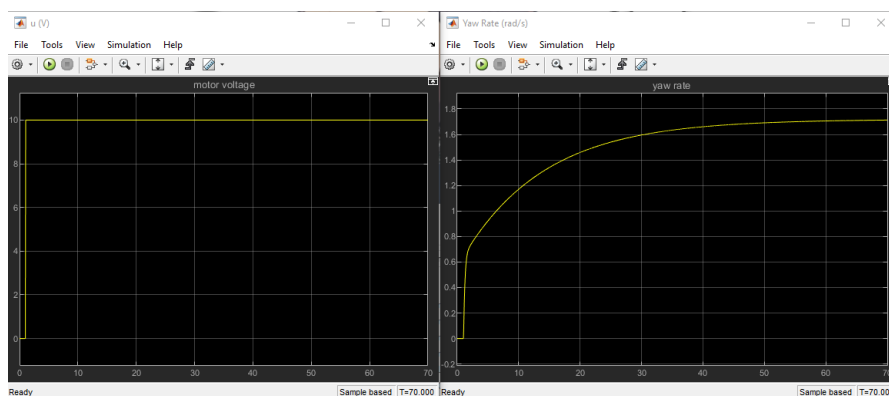


Figure 15: Réponse à un échelon de l'Aero

- Déterminez le gain en régime permanent K à l'aide de la réponse à l'échelon mesurée et de l'équation 11. Utilisez l'outil **Cursor Measurements** dans le Scope de Simulink pour effectuer des mesures directement sur la courbe de réponse, comme illustré dans la Figure 16.

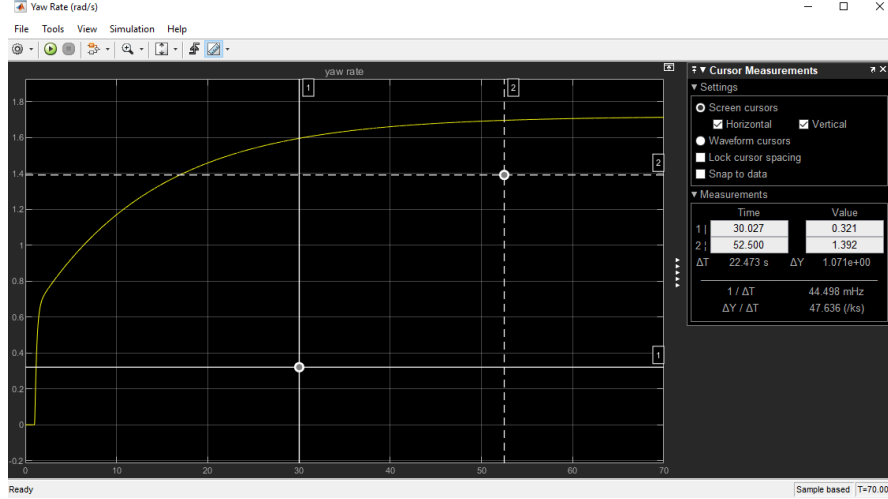


Figure 16: Déterminez le gain en régime permanent K à l'aide de la réponse à l'échelon mesurée.

- Déterminez la constante de temps T à partir de la réponse obtenue et de l'équation 13.
- Calculez le gain de force de poussée K_f et l'amortissement visqueux D_y du modèle de fonction de transfert de la vitesse angulaire de lacet du demi-quadrotor à partir du gain en régime permanent K et de la constante de temps T mesurés, ainsi que des paramètres du modèle Aero, en utilisant les équations suivantes :

$$T = \frac{J_y}{D_y} \quad (14)$$

$$K = \frac{D_t K_f}{D_y} \quad (15)$$

- Pour vérifier si les paramètres de modèle K_f et D_y que vous avez dérivés sont corrects, ouvrez le fichier `/Software/Quanser_Aero/Parameter_Estimation/q_aero2_half_quad_model_val.slx` et remplacez les notations dans le bloc de fonction de transfert par vos paramètres. Le modèle est présenté dans la Figure 17.

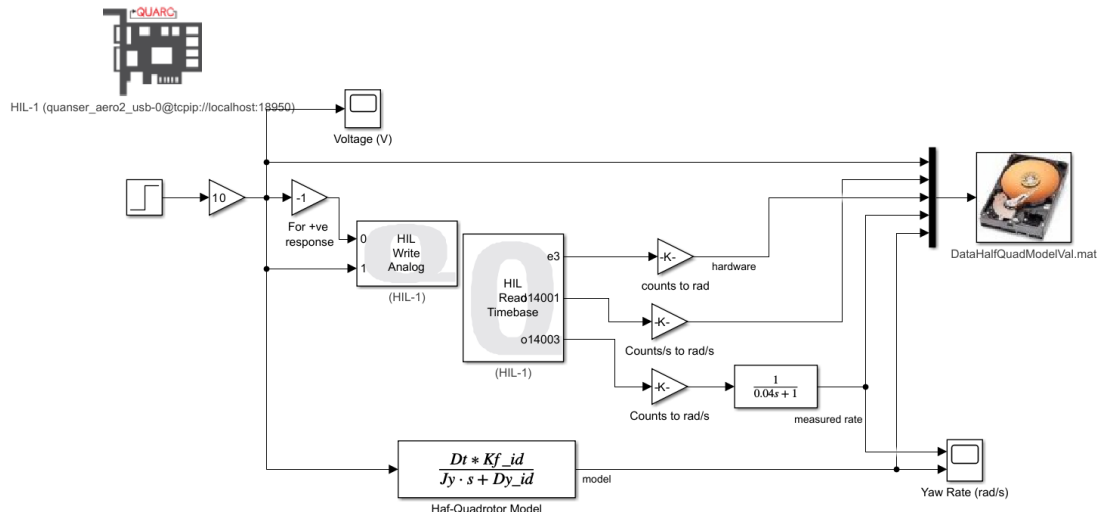


Figure 17: Modèle Simulink pour la détermination des paramètres du modèle.

8. Évaluez la réponse de la courbe du taux de lacet et expliquez.

2.2. Analyse de la stabilité du modèle

Analysez la stabilité du système en répondant aux questions suivantes :

1. Déterminez la stabilité du système Aero tension-position à partir de ses pôles.
2. Déterminez la stabilité du système servo tension-vitesse à partir de ses pôles.
3. Appliquez une tension échelon unitaire à l'Aero en exécutant le modèle Simulink. Représentez la réponse indicielle de la position, de la vitesse, ainsi que la tension appliquée au moteur.
4. En vous basant sur la réponse en vitesse et le principe EBSB (entrée bornée, sortie bornée), quelle est la stabilité du système ? Comment cela se compare-t-il à vos résultats de l'analyse des pôles ?
5. En vous basant sur la réponse en position et le principe EBSB, quelle est la stabilité du système ? Comment cela se compare-t-il à vos résultats de l'analyse des pôles ?
6. Existe-t-il une entrée pour laquelle la réponse en position en boucle ouverte du système Aero est stable ? Si oui, modifiez le diagramme Simulink pour inclure votre entrée, testez-la sur l'Aero et montrez la réponse en position. D'après ce résultat, comment pourriez-vous définir la stabilité marginale en termes d'entrées bornées ?

2.3. Conception et implementation d'un correcteur de type PID

Les exigences de performance pour la commande de la position angulaire sont :

1. Suivi du point de consigne de position → aucune erreur en régime permanent
2. Tension d'entrée du moteur limitée à $[-24V; +24V]$
3. Dépassement → $OS = 2.5\%$
4. Peak time 3.5 → $T_p = 3.5$
5. Rejet des perturbations → rejet des effets de charge

2.3.1. Rappel

Une variation du correcteur PID classique sera utilisée comme montré dans la Figure 18 et dans la structure suivante :

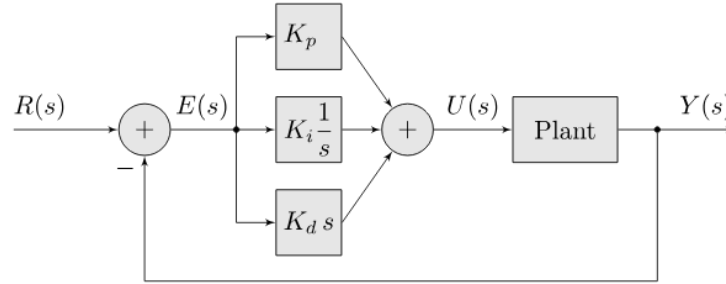


Figure 18: PID

$$u(t) = K_p(r(t) - y(t)) - K_d \cdot \dot{y}(t) + K_i \int (r(t) - y(t)) dt \quad (16)$$

où K_p est le gain proportionnel, K_i le gain intégral, K_d le gain dérivé. $r = \psi_d(t)$ est la commande de référence en lacet, $y = \psi(t)$ est l'angle de lacet mesuré, et $u = V(t)$ est l'entrée de contrôle (tension appliquée au moteur).

$$U(p) = K_p(R(p) - Y(p)) - K_d p Y(p) + K_i \frac{1}{p} (R(p) - Y(p)) \quad (17)$$

À partir de l'équation 8 :

$$\frac{Y(p)}{U(p)} = \frac{\psi(p)}{U(p)} = \frac{D_t K_f}{p (J_y p + D_y)} \Rightarrow U(p) = \frac{p Y(p) (J_y p + D_y)}{D_t K_f}$$

En substituant dans l'équation 17, on obtient :

$$Y(p) = \frac{D_t K_f}{p(D_y + J_y \cdot p)} \left[K_p (R(p) - Y(p)) - K_d p Y(p) + K_i \frac{1}{p} (R(p) - Y(p)) \right] \quad (18)$$

En résolvant pour $\frac{Y(p)}{R(p)}$, on obtient l'expression en boucle fermée :

$$\frac{Y(p)}{R(p)} = \frac{D_t K_f K_p p + D_t K_f K_i}{J_y p^3 + (D_t K_f K_d + D_y) p^2 + D_t K_f K_p p + D_t K_f K_i} \quad (19)$$

Il s'agit d'une fonction de transfert du second ordre. Rappelons la fonction de transfert standard du second ordre :

$$\frac{Y(p)}{R(p)} = \frac{\omega_n^2}{p^2 + 2\xi\omega_n p + \omega_n^2} \quad (20)$$

Supposons que $K_i = 0$:

$$\frac{Y(p)}{R(p)} = \frac{D_t K_f K_p}{J_y p^2 + (D_t K_f K_d + D_y) p + D_t K_f K_p} = \frac{J_y \cdot \omega_n^2}{J_y p^2 + 2J_y \xi \omega_n p + J_y \omega_n^2} \quad (21)$$

2.3.2. Implémenter un correcteur de type PID pour le système Aero

Pour synthétiser et implémenter un correcteur de type PID pour le système Aero, suivez les étapes suivantes :

1. Déterminez la fréquence naturelle ω_n et le coefficient d'amortissement ξ correspondant aux exigences issues du dépassement en pourcentage et du temps de stabilisation, selon les équations :

$$OS = 100e^{\left(\frac{-\pi\xi}{\sqrt{1-\xi^2}}\right)} \quad \text{et} \quad T_p = \frac{\pi}{\omega_n \sqrt{1-\xi^2}} \quad (22)$$

2. Déterminez les valeurs de K_p , K_i et K_d basées sur l'équation 21 qui permettent à la réponse indicielle de la fonction de transfert en boucle fermée d'atteindre le dépassement en pourcentage et le temps de stabilisation requis. Adaptez les valeurs obtenues lors de la séance de résolution de problèmes à votre modèle identifié.
3. Vérifiez la stabilité du système en boucle fermée, contrôlez le gain du régulateur et analysez également les marges de stabilité du système en boucle ouverte.
4. Ouvrez dans Simulink le fichier `/Software/Quanser_Aero/PID_control/q.aero2_half_quad_pd.slx` in simulink. Réglez les valeurs de K_p , K_i et K_d selon vos paramètres. Le modèle est présenté dans la Figure 19

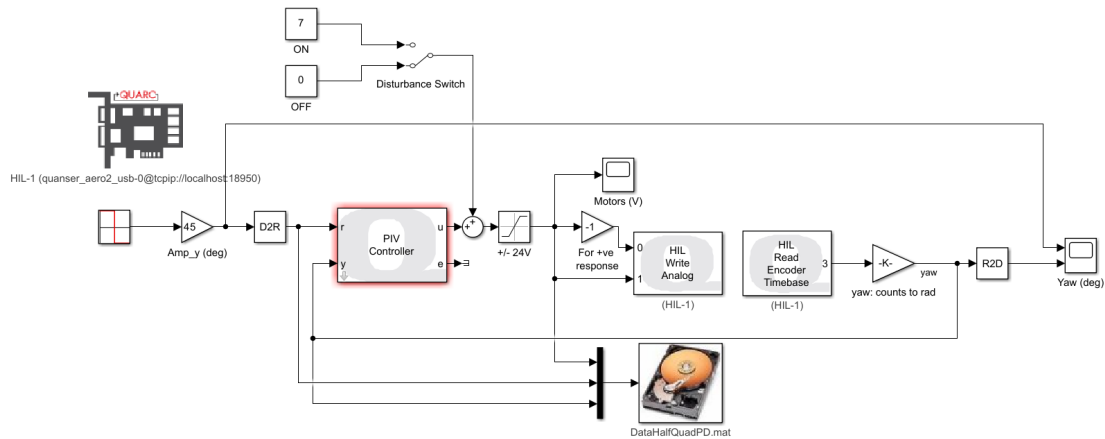


Figure 19: PID Simulink

5. Cliquez sur **Monitor & Tune** dans le panneau Hardware pour exécuter le contrôleur.
6. Observez la réponse de la position de l'angle de lacet à une consigne en échelon carré ainsi que le signal de commande.
7. La réponse expérimentale est-elle proche des résultats de la simulation ?
8. Les spécifications des exigences sont-elles remplies ? Sinon, modifiez les gains du contrôleur PID K_p , K_i et K_d afin d'obtenir la meilleure réponse aux exigences (dépassement, temps de pic, erreur en régime permanent et amplitude de la tension du moteur).
9. Déterminez le temps de stabilisation t_s .
10. Remplacez la consigne par une onde sinusoïdale au lieu de l'échelon carré et répétez l'expérience précédente. Le contrôleur PID peut-il suivre la référence sinusoïdale ?
11. Arrêtez le contrôleur QUARC.

2.3.3. Test de robustesse du correcteur PID de l'Aero

Testez la robustesse du correcteur synthétisé, analysez les résultats et commentez :

1. Supposons qu'une tension de perturbation simulée constante $V_{sd} = 7\text{ V}$ soit ajoutée à la tension du moteur pour simuler une perturbation. Les spécifications des exigences sont-elles remplies ? Sinon, modifiez les gains, K_p , K_i et K_d , du correcteur PID afin d'obtenir la meilleure réponse aux spécifications.
2. Observez la réponse de la position de lacet à la consigne en échelon carré ainsi que le signal de commande.
3. Arrêtez le contrôleur QUARC.

2.4. Conception d'un correcteur par placement des pôles de la boucle fermée, structure RST

Les exigences de performance pour la commande de la position angulaire sont :

1. Suivi du point de consigne de position → aucune erreur en régime permanent
2. Tension d'entrée du moteur limitée à $[-24V; +24V]$
3. Dépassement → $OS = 2.5\%$
4. Peak time 3.5 → $T_p = 3.5$
5. Rejet des perturbations → rejet des effets de charge

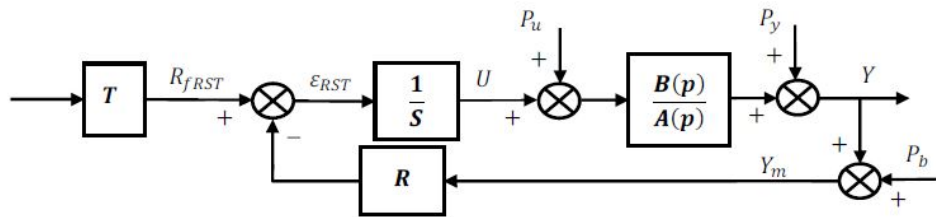


Figure 20: Structure RST (cours AU360, E. Mendes).

La sortie Y , la position de lacet, et la commande U , la tension du moteur, peuvent s'exprimer en fonction des polynômes des fonctions de transfert et des entrées exogènes:

$$Y = \frac{1}{A \cdot S + B \cdot R} (B \cdot T \cdot Ref + B \cdot S \cdot P_u + A \cdot S \cdot P_y - B \cdot R \cdot P_b) \quad (23)$$

$$U = \frac{1}{S} (T \cdot Ref - R \cdot (Y + P_b)) \quad (24)$$

Les relations (24) montre que les pôles de la boucle fermée sont les racines du polynôme:

$$D = A \cdot S + B \cdot R \quad (25)$$

1. Considérer la fonction de transfert pour la position en lacet (8) et déterminer le degré des polynômes $R(p)$, $S(p)$ et $T(p)$.
2. Proposer une forme pour le polynôme $D(p)$ et déterminer son degré.
3. Construire le système linéaire pour résoudre $D = A \cdot S + B \cdot R$ et déterminer les coefficients des polynômes $R(p)$, $S(p)$ et $T(p)$.
4. Analyser le comportement du système en boucle fermée y compris les marges de stabilité.
5. Implémentez votre correcteur RST en simulation, puis sur le système réel. Procédez à sa discrétisation et implémentez le contrôleur RST conçu sur Arduino.

3. Implémentation de la commande sur Arduino

3.1. Préparation

Pour programmer l'Arduino, vous devez préparer les éléments suivants : une carte Arduino Uno, un câble de transfert de données, une interface SPI, et le logiciel de programmation Arduino IDE. Vous pouvez connecter l'Arduino Uno à l'interface SPI comme montré dans la Figure 21.

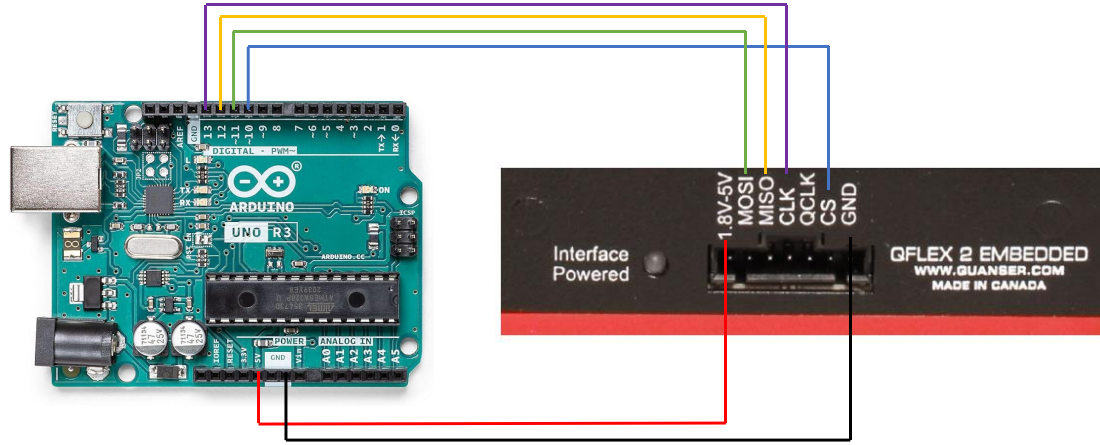


Figure 21: Wiring arduino uno to SPI interface

3.2. Discrétisation du correcteur

Pour convertir le contrôleur du domaine continu au domaine discret, utilisez la discrétisation d'Euler suivante :

$$\dot{x}(t) = \frac{x(k) - x(k-1)}{T_s} \quad (26)$$

Pour augmenter la stabilité du signal de commande, nous allons utiliser un filtre passe-haut :

$$F(p) = \frac{\omega_n p}{p + \omega_n}$$

En appliquant le filtre passe-haut à la partie dérivée de l'équation PID de Laplace 17, nous obtenons :

$$\begin{cases} U(p) = K_p(R(p) - Y(p)) + K_d F(p)(R(p) - Y(p)) + K_i \frac{1}{p}(R(p) - Y(p)) \\ R(p) - Y(p) = E(p) \quad (\text{erreur en domaine de Laplace}) \end{cases}$$

$$\Rightarrow U(p) = K_p \cdot E(p) + K_d F(p)E(p) + K_i \frac{1}{p}E(p)$$

Supposons que $K_i = 0$:

$$U(p) = K_p \cdot E(p) + K_d F(p)E(p). \quad (27)$$

Soit $E_f(s)$ le *error_filter* :

$$E_f(p) = F(p)E(p) = \frac{\omega_n p}{p + \omega_n} E(p)$$

$$\Leftrightarrow pE_f(p) + \omega_n E_f(p) = \omega_n pE(p)$$

Retour au domaine temporel :

$$\dot{e}_f(t) + \omega_n e_f(t) = \omega_n \dot{e}(t) \quad (28)$$

L'équation discrète 28, basée sur l'équation 26, est la suivante :

$$\begin{aligned} \frac{e_f(k) - e_f(k-1)}{T_s} + \omega_n e_f(k) &= \omega_n \cdot \frac{e(k) - e(k-1)}{T_s} \\ \Rightarrow e_f(k) &= \frac{\omega_n [e(k) - e(k-1)] + e_f(k-1)}{1 + \omega_n T_s} \end{aligned}$$

Soit $\alpha = \frac{\omega_n T_s}{1 + \omega_n T_s}$, alors nous avons :

$$\Rightarrow e_f(k) = \alpha \cdot \underbrace{\frac{[e(k) - e(k-1)]}{T_s}}_{\text{derivative-error}} + (1 - \alpha)e_f(k-1) \quad (29)$$

L'équation discrète 27 pour la programmation Arduino est :

$$(27) \Leftrightarrow U(p) = K_p E(p) + K_d E_f(p)$$

$$\Leftrightarrow u(k) = K_p e(k) + K_d e_f(k) \quad (30)$$

Dans le programme Arduino, nous utilisons l'équation 29 et l'équation 30 pour calculer le signal d'entrée du Quanser Aero.

3.3. Programmation du correcteur

Téléchargez le dossier **PID_quanser_aero_arduino** dans **/Software/Quanser_Aero**, ce dossier contient 1 fichier Arduino et 4 fichiers de bibliothèque.

Ouvrez le fichier Arduino **PID_quanser_aero_arduino.ino** dans le logiciel de programmation Arduino IDE, le code est présenté en Annexe A.1.

3.3.1. Explication du code

1. **Lignes 1 à 5** : Déclarez les bibliothèques nécessaires.
2. **Lignes 7 à 19** : Cette partie explique qu'il existe des variables globales déjà définies dans les fichiers de bibliothèque. Vous pouvez utiliser ces variables sans les redéfinir.

3. **Lignes 21 à 25 :** Vous pouvez définir vos propres variables dans cette partie.
4. **Lignes 27 à 36 :** Cette section montre les configurations initiales lorsque le code commence à s'exécuter, y compris : la fonction de configuration du PIN esclave, l'initialisation de l'interface SPI, l'initialisation de la communication série.
5. **Lignes 40 à 73 :** La fonction `loop` est la fonction principale qui s'exécute lorsque le code tourne.
6. **Lignes 45 à 49 :** Cette section réinitialise le système avant d'exécuter les autres fonctions. Aucune modification n'est nécessaire.
7. **Lignes 51 à 54 :** Initialisation de l'interface SPI pour la transmission des données.
8. **Lignes 56 à 57 :** Appel de la fonction `readSensors()`. Cette fonction vous permet d'obtenir la position angulaire de lacet (yaw) du Quanser Aero à partir du capteur encodeur. La variable globale de l'angle est : `yaw`.
9. **Lignes 65 à 66 :** Appel de la fonction `driveMotor()`. Après avoir calculé le signal d'entrée pour le QUBE Servo, stockez-le dans les variables globales `motor0Voltage` et `motor1Voltage`. La fonction `driveMotor()` fera fonctionner les rotors.
10. **Lignes 69 à 72 :** Met en pause le système pendant un temps d'échantillonnage, désélectionne le PIN esclave et termine la transaction SPI.

3.3.2. Implémentation du contrôleur PID sur Arduino

Notez qu'il est important de vérifier d'abord le code dans le logiciel Arduino IDE avant de le téléverser sur la carte Arduino.

Pour implémenter le contrôleur PID sur Arduino, les étapes principales à suivre sont les suivantes :

1. Dans la fonction `loop()`, après la ligne 42, écrivez un morceau de code pour changer `psi_desired` en $-psi_desired$ après 3000 boucles. Fixez `psi_desired` à $\frac{\pi}{6}$.
2. Après la ligne 59, écrivez un correcteur PID basé sur l'équation 29 et l'équation 30. Soit $\omega_n = 75$.
Le signal d'entrée calculé est U , avec $motor0Voltage = -U$ et $motor1Voltage = U$.
3. Après la ligne 62, écrivez un morceau de code pour saturer le signal d'entrée entre -24 et 24 .
4. Afficher les variables dans Arduino plotter : temps d'échantillonnage, la position en lacet et la position désirée. Analyser.

5. Implémentez l'algorithme du contrôleur à l'intérieur d'une routine d'interruption (ISR-interrupt service routine) afin de séparer les tâches critiques (comme les calculs de contrôle) des tâches moins critiques (comme l'affichage des variables). Cette approche permet d'éviter les retards dans l'exécution du contrôleur, garantissant que la boucle de contrôle fonctionne avec une synchronisation précise et une fiabilité améliorée.

A. Annexes

A.1. Arduino code

```

1 #include <SPI.h>
2 #include <math.h>
3 // Quanser Aero library
4 #include "Aero.h"
5 #include "ACSI_aero-lib.h"
6
7 /*                                IMPORTANT INFORMATION
8
9 Accessible global variables (just use them):
10 These are the variables that already be defined in QUBE Servo library ,
11 you don't need to define them again in this file
12
13     - Input variables (you can get them after readSensors() function):
14         float yaw;           // yaw angle position in radians
15
16     - Output variables (you can send them by driveMotor() function):
17         float motor0Voltage; // Signal sent to the motor 0 in Volts
18         float motor1Voltage; // Signal sent to the motor 1 in Volts
19 */
20
21 /* DECLARE GLOBAL VARIABLES HERE IF NEEDED */
22 float Ts = 0.002; // second
23
24
25 /*-----*/
26
27 //This function will be called once during initialization
28 void setup() {
29     // Set the slaveSelectPin as an output
30     pinMode(slaveSelectPin , OUTPUT);
31
32     // Initialize SPI
33     SPI.begin();
34
35     // Initialize serial communication at 250000 baud
36     Serial.begin(250000);
37
38 }
39
40 // This function will be called repeatedly until Arduino is reset
41 void loop() {
42     // After 3000 loops , change the theta_desired = -theta_desired (for
43     // students)
44
45     // Reset after the Arduino power is cycled or the reset pushbutton is
46     // pressed
47     if (startup) {
48         resetQuanserAero();
49     }
50 }

```

```
48     startup = false;
49 }
50
51 // initialize the SPI bus using the defined speed, data order and data
52 // mode
53 SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE2));
54 // take the slave select pin low to select the device
55 digitalWrite(slaveSelectPin, LOW);
56
57 // read the sensors
58 readSensors();
59
60 // PID Controller (for students)
61
62 // The variable "motor0Voltage" and "motor1Voltage" is saturated within
63 // the range of -24V to 24V. (for students)
64
65 // drive motor
66 driveMotor();
67
68
69 delay(Ts*1000);
70 // take the slave select pin high to de-select the device
71 digitalWrite(slaveSelectPin, HIGH);
72 SPI.endTransaction();
73 }
```

Listing 1: PID_quanser_aero_arduino.ino